



Actividad 3.7. Jugando con botones

Vamos a crear un pequeño juego en el que esconderemos la respuesta correcta debajo de alguno de los botones. En esta ocasión vamos a ver cómo **añadir dinámicamente una lista de botones a un Scroll View**. Dentro del **Scroll View** tendremos un **Linear Layout** llamado **lBotonera** donde colocaremos los botones.

Tendremos que:

- Crear el proyecto.
- Establecer en el `activity_main.xml` tanto el `ScrollView` como el `Linear Layout`.
- Definir el número de botones
- Obtener la referencia al `Linear Layout lBotonera` donde colocar los botones.
- Iterar en un bucle creando tantos objetos `Button` como queramos
- Asignar propiedades de `LinearLayout` al botón o bien usar sus propiedades de ancho y alto.
- Añadir el botón a la botonera.
- Darle vida a los botones.
- Arreglos de última hora.
- Compilar y probar.

Vamos a ver el código necesario.

1. Primero crea un nuevo proyecto en Android Studio del tipo “Empty Views Activity”

Llámale “**Juego_botones**”, el Lenguaje “Kotlin” y aunque tenemos diversas plataformas y APIs en las que utilizar nuestra aplicación, nosotros nos centraremos en aplicaciones para teléfonos y tablets, en cuyo caso tan sólo tendremos que seleccionar la API mínima (es decir, la versión mínima de Android) que soportará la aplicación. Android 9.0. como versión mínima (API 28):

Una vez configurado todo pulsamos el botón **Finish** y Android Studio creará por nosotros toda la estructura del proyecto y los elementos indispensables que debe contener. Si todo va bien aparecerá la pantalla principal de Android Studio con el nuevo proyecto creado.

2. Creando el layout

Simplemente tendremos que añadir un ScrollView que se ajuste a todo el alto y ancho de la pantalla y dentro un LinearLayout vertical donde meteremos los botones dinámicamente. Y ya hemos acabado con el activity_main.xml

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:tools="http://schemas.android.com/tools"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:orientation="vertical">

    <ScrollView
        android:layout_width="match_parent"
        android:layout_height="match_parent">
        <LinearLayout
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:orientation="vertical"
            android:id="@+id/llBotonera" >

            </LinearLayout>
        </ScrollView>
    </LinearLayout>
```

3. Definición del número de botones que queremos en el juego

Declaramos una constante llamada numBotones con el número que queramos, en mi caso 10.

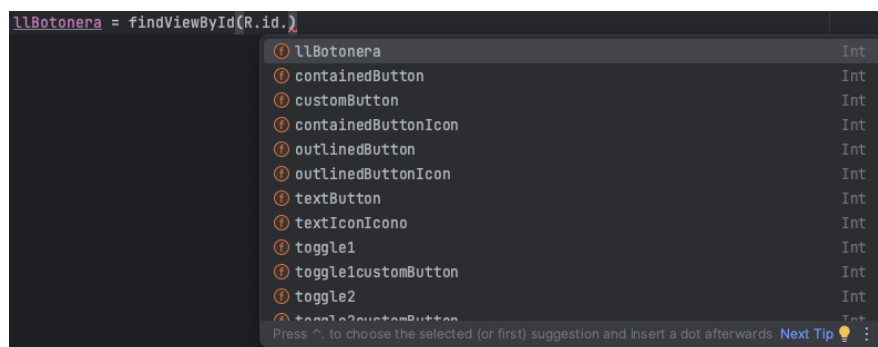
```
class MainActivity : AppCompatActivity() {
    private val numBotones = 10
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContentView(R.layout.activity_main)
    }
}
```

4. Referenciar al LinearLayout

Para acceder a cualquier View, tendremos que utilizar el método “findViewById”. Recordemos que definimos el identificador “llBotonera” en el activity_main.xml

```
<LinearLayout  
    android:layout_width="match_parent"  
    android:layout_height="wrap_content"  
    android:orientation="vertical"  
    android:id="@+id/llBotonera" >  
  
</LinearLayout>
```

Utilizaremos el método findViewById que nos permite, a través de la clase R (clase para acceder a todos los recursos) acceder a todos los identificadores que tenemos definidos en la app.



```
class MainActivity : AppCompatActivity() {  
    private val numBotones = 10  
    private lateinit var llBotonera: LinearLayout  
    override fun onCreate(savedInstanceState: Bundle?) {  
        super.onCreate(savedInstanceState)  
        setContentView(R.layout.activity_main)  
  
        llBotonera = findViewById(R.id.llBotonera)  
    }  
}
```

Si queremos definir la variable en la clase, tendremos que advertirle a Kotlin que iniciaremos la variable más tarde con la palabra reservada “lateinit”. Para acceder al identificador del Layout

5. Creación dinámica de botones

Ahora vamos a crear un bucle para crear los botones dinámicamente. En primer lugar creamos **lp**, que va a contener la configuración por defecto del layout de todos los botones. Son `LinearLayout.LayoutParams` porque los botones van a estar en un `LinearLayout`.

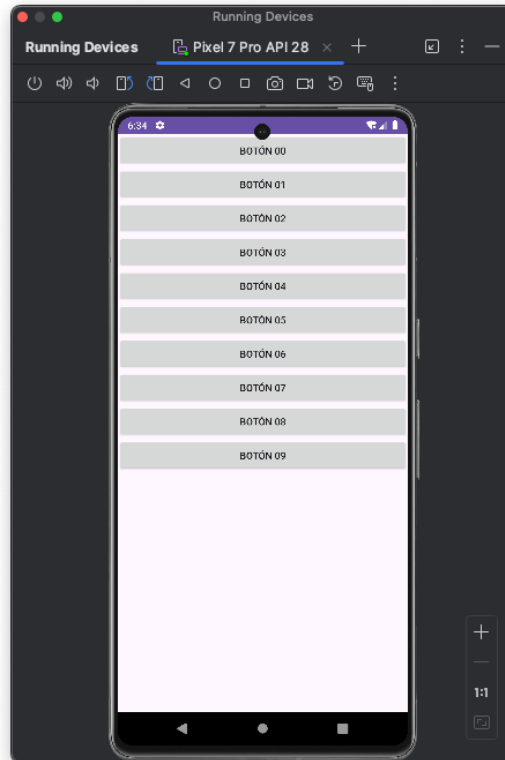
Ahora vamos a crear el bucle, desde `i = 0` hasta `numBotones`. Vemos que en cada iteración creamos el botón, le asignamos las propiedades que hemos definido en `lp`, le ponemos un texto según la iteración del bucle y finalmente lo añadimos al `View llBotonera` con el método `.addView`

```
class MainActivity : AppCompatActivity() {  
    private val numBotones = 10  
    private lateinit var ll Botonera: LinearLayout  
    override fun onCreate(savedInstanceState: Bundle?) {  
        super.onCreate(savedInstanceState)  
        setContentView(R.layout.activity_main)  
  
        llBotonera = findViewById(R.id.llBotonera)  
  
        //Creamos las propiedades de layout que tendrán los botones.  
        val lp = LinearLayout.LayoutParams(  
            LinearLayout.LayoutParams.MATCH_PARENT,  
            LinearLayout.LayoutParams.WRAP_CONTENT  
        )  
        //Creamos los botones en bucle  
        for (i in 0 until numBotones) {  
            val button = Button(this)  
            //Asignamos propiedades de layout al botón  
            button.layoutParams = lp  
            //Asignamos Texto al botón  
            button.text = "Botón " + String.format("%02d", i)  
            llBotonera.addView(button)  
        }  
    }  
}
```

1.

6. Dando una utilidad a los botones

Aunque el código en este punto, debería funcionar y crear los botones, vemos que pese a poder pulsarlos, no hacen nada. Te debería salir algo así:



Vamos a añadir en la creación de cada botón, dentro del bucle, la llamada al método `setOnClickListener` para cada uno de los i botones.

```
//Creamos los botones en bucle
for (i in 0 until numBotones) {
    val button = Button(this)
    //Asignamos propiedades de layout al botón
    button.layoutParams = lp
    //Asignamos Texto al botón
    button.text = "Botón " + String.format("%02d", i)
    //Asignamos el Listener
    button.setOnClickListener(buttonClickListener(i))
    //Añadimos el botón a la botonera
    llBotonera.addView(button)
}
```

Como vemos estamos uniendo el método `setOnClickListener` de la clase `button` con la llamada a una función privada que tenemos que declarar. Esta función la tenemos que crear dentro de la clase `MainActivity`. (Cuidado no la crees dentro de la función `onCreate` donde hemos añadido el bucle)

```
private fun buttonClickListener(index: Int): View.OnClickListener {
    return View.OnClickListener {
        Toast.makeText(this@MainActivity,
            "Botón " + String.format("%02d", index) + " pulsado.",
            Toast.LENGTH_SHORT).show()
    }
}
```

Lo que estamos realizando simplemente es que cuando pulsemos uno de los botones, nos muestre un aviso o **"Toast"** con el mensaje "Botón XX pulsado.". Los avisos solo ocupan la cantidad de espacio necesario para el mensaje, y la actividad en curso permanece visible y admite la interacción. Los avisos desaparecen automáticamente después de un tiempo de espera.



Las duraciones de los avisos pueden ser cortas "LENGTH_SHORT" (2 segundos) o largas "LENGTH_LONG" (3,5 segundos) y no se pueden modificar ya que están pensadas para avisos puntuales. Si necesitamos un mensaje más largo tendremos que utilizar los mensajes de la barra de estado "NotificationCompat"

7. Arreglos de última hora

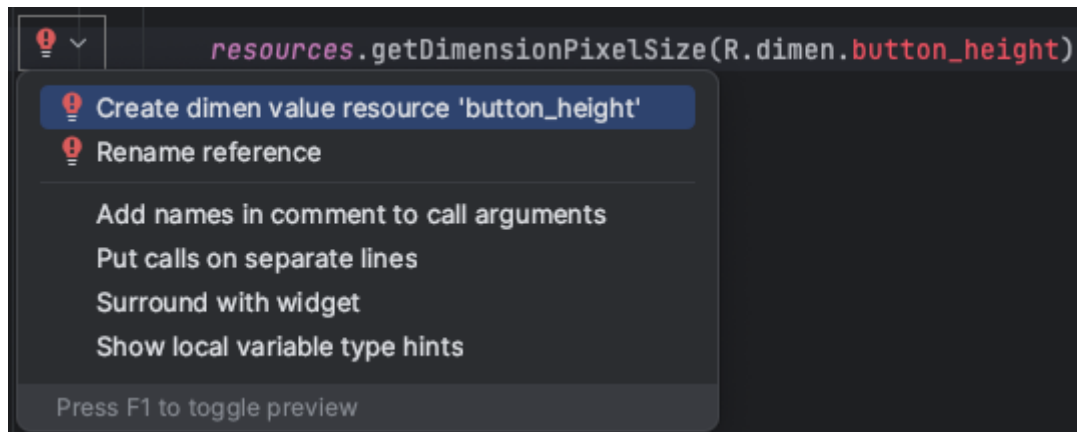
Uno de los arreglos que vamos a hacer es hacer que los botones que se crean en el bucle, sean más altos, de manera que los 10 botones ocupen mayor espacio que la ventana y así poder ver el ScrollView en funcionamiento. Para conseguir esto necesitamos cambiar los LayoutParams y añadir un valor concreto.

```
val lp = LinearLayout.LayoutParams(  
    LinearLayout.LayoutParams.MATCH_PARENT,  
    resources.getDimensionPixelSize(R.dimen.button_height)  
)
```

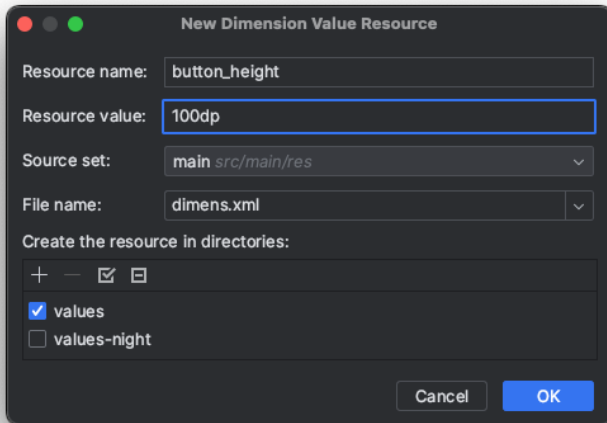
El valor de esta constante lo vamos a obtener dinámicamente, pero con un valor que nosotros queramos. Algo parecido al archivo de colores o el de strings.

```
resources.getDimensionPixelSize(R.dimen.button_height)
```

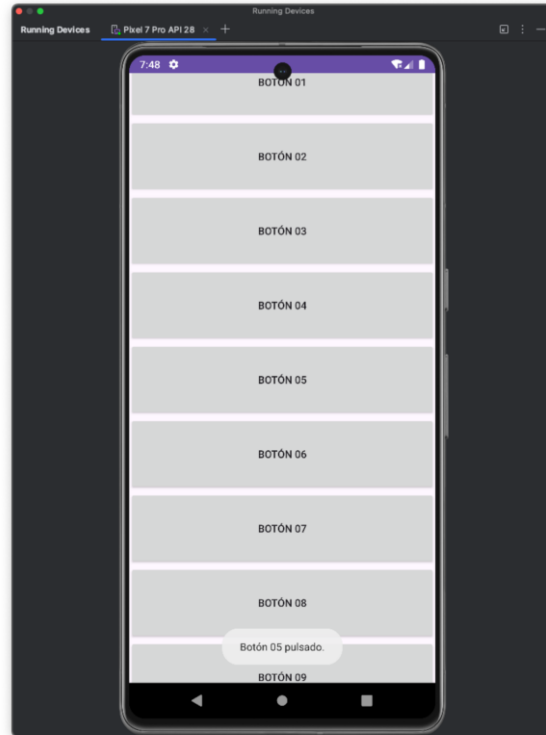
Primero accederemos al método **getDimensionPixelSize** disponible en la clase resources que devuelve un valor de dimensión. En este caso, definiremos un atributo de dimensión en el archivo **dimens.xml**. Este no es tan accesible como los de colores o strings, pero si le damos un nombre (**button_height**), nos mostrará un error y nos sugerirá añadirlo al **dimens.xml**.



Si le damos a crear el atributo, nos saldrá una ventana igual que cuando definimos strings. Añadiremos el valor 100dp (100 Dots Per inch)



8. Compilar y probar el funcionamiento de la App en un dispositivo virtual



Se puede comprobar que los botones son más altos, y que no caben en la pantalla. Podemos ver como el Scroll está desplazado y que hemos pulsado el botón 5, como muestra el aviso.

9. Ahora tu parte

Modifica un poco el código que tienes hasta ahora y que la aplicación haga lo siguiente.

1. Que **solo uno** de los botones muestre un aviso que ponga `"ME ENCONTRASTE!!"`
 2. Que en **el resto** de botones muestre un aviso que ponga `"Sigue buscando"`
 3. Y que el botón donde se encuentra el mensaje único sea **aleatorio** cada vez.
- 3.
 4. Crea el .apk y entrégalo en Moodle.

10. Ejemplo

